LocalGPT documentation

Version 2.3.3

Tracked source documentation snapshot. The owner-side release build replaces this file with the complete DocFX HTML-backed PDF and generated API reference.

index.md

LocalGPT documentation

Version 2.3.3

LocalGPT is a local-first AI workbench for direct chat, configurable AI Councils, project maintenance, persistent knowledge, provider-qualified model routing, embedded planning, game runtimes, and guarded local execution.

This site is the maintained product and architecture documentation. It deliberately separates stable design from historical experiments, so you can understand *what LocalGPT is now* without walking through every learning step that shaped it. ■

Choose a path

■ *Use LocalGPT*

Start with the application concepts, common workflows, and the boundaries around actions that can change files, run tools, or operate hardware.

Open the user guide (guide/index.md)

■ *Understand the architecture*

Follow the modular-monolith boundaries from Blazor UI through services, provider adapters, persistence, the AI Host control plane, Council orchestration, and 1-Wire transports.

Explore the architecture (architecture/index.md)

■■ *Build and maintain it*

Read the compilation truth, diagnostics policy, documentation pipeline, release checks, and the rules that keep generated work reviewable.

Open engineering guidance (engineering/index.md)

■ *Look up details*

Use the capability map, design evolution notes, documentation migration map, or the complete XML-generated API reference.

Browse reference material (reference/index.md)

Architecture at a glance

```
flowchart LR
    U[Human user] --> UI[Blazor + DevExpress UI]
    UI --> APP[Application services]
    APP --> COUNCIL[Chat and Council runtime]
    APP --> PROJECTS[Projects, workspaces and knowledge]
    APP --> HOST[Provider-qualified AI Host control plane]
    APP --> WIRE[Organic 1-Wire boundary]
    COUNCIL --> PROVIDERS[Ollama / OpenAI-compatible / cloud providers]
    PROJECTS --> DB[(SQLite + EF Core)]
    HOST --> PROVIDERS
    WIRE --> PEERS[Approved peers, publishers and devices]
```

The human request remains the authority. Model output, uploaded content, repository text, remote peers, and generated artifacts are data—not permission.

Complete documentation set

The conceptual pages are built together with compiler-generated XML documentation for public types and members. The same themed HTML site is shipped inside LocalGPT and published to GitHub Pages.

The packaged PDF is a curated handbook: it contains the maintained guide, architecture, engineering, and reference chapters plus a linked inventory of every generated API page. A normal owner-side documentation build can additionally render the complete API body into the PDF.

■ Download the Kawaii handbook

guide/chat-and-council.md

Chat and AI Council

Direct chat

The Chat page is the primary conversation surface. One provider-qualified model answers ordinary requests while configuration remains collapsible so the transcript keeps the available viewport.

A chat session owns its selected provider route, protocol profile, formatter, bounded context, and feedback state. Protocol-specific control tokens are normalized by the matching profile instead of being scattered through the UI.

Supported protocol families include Harmony-style channels, DeepSeek/R1 thinking markers, Gemma turn markers, Apple/OpenELM-style roles, and generic think-tag formats. The formatter boundary keeps these families isolated.

Council teams

A Council team turns one request into an explicit workflow. Each member has a role, provider-qualified model route, optional runtime class, and bounded task. The team configuration is durable; the active run state is not replaced by whichever model speaks last.

Typical roles include:

- coordinator or leader;
- implementation specialist;
- architecture or safety reviewer;
- verifier;
- domain specialist;
- world actor for GameDirector sessions;
- human participant when invited.

The Council spooler and preflight services prepare work, validate routes, and track steps. A model can propose a result, but deterministic application rules still decide whether a step is complete, failed, recoverable, or waiting for a human.

Provider-qualified routing

The visible address follows this shape:

```
Provider — model @ endpoint
```

A bare model name is accepted only when it resolves to one provider route. Ambiguous or stale selections fail clearly instead of falling back to an arbitrary endpoint.

Mixed-provider Councils can combine Ollama, LM Studio or another OpenAI-compatible endpoint, OpenAI, and Azure OpenAI. The correct client is created immediately before each call. Provider-specific settings—such as Ollama GPU options—stay with the provider that understands them.

Benchmark Council

A bounded benchmark measures one or more selected routes using deterministic tasks and explicit limits for context, output, count, and timeout. Other selected models can review the recommendation; when no independent reviewer is

available, a bounded self-review may be used.

Recommendations are never applied automatically. The user chooses whether to save or apply a preset. Applying a preset changes future LocalGPT routing; it does not rewrite provider-global server configuration.

Result integrity

A step that produces thinking but no substantive final answer receives at most one bounded final-only recovery request. If the result is still empty, the step is failed rather than counted as success.

Peer verification is advisory. A failed verifier can be recorded as a warning while a valid result remains available, but the UI must not relabel unrelated notices as successful verification.

Human participation

A human can join a running Council, answer a question, approve a proposed boundary, or provide a correction. Human participation does not turn every human message into unrestricted authority; the application still binds decisions to the current request, action, target, and approval scope.

DX functions and change reviews

DX functions expose bounded application capabilities through typed parameters and explicit handlers. Generated changes can be stored as review records with proposed files, findings, execution status, and human decisions. The function catalog describes what can be requested; it does not bypass the service that owns the real operation.

> [!NOTE]

> Council output is a structured proposal pipeline, not a magic quorum. The boring deterministic parts are what keep the fun parts usable.

guide/documentation.md

Documentation inside LocalGPT

One documentation tree

The conceptual documentation and compiler-generated API reference are built into one DocFX site. That exact static tree is:

• copied into wwwroot/help-docs for the running application;

• packaged inside release ZIPs;

• extracted by the GitHub Pages workflow;

• used to generate the versioned PDF or packaged handbook.

GitHub Pages does not rebuild LocalGPT or require DevExpress packages. It publishes the already reviewed documentation shipped in a release.

Important routes

• /help-docs/index.html — documentation home in the local application.

• /help-docs/api/index.html — generated API reference.

• /api/documentation/status — build mode, counts, PDF state, and theme hashes.

• /api/documentation/comments — bounded XML-comment search.

• /api/documentation/pdf — current versioned PDF.

On GitHub Pages, the equivalent root is /LocalGPT/.

Theme selector

The navigation theme menu supports Light, Dark, and Auto. The selected value is written to:

• DocFX's theme local-storage key for compatibility;

• LocalGPT's localgpt-docs-theme local-storage key;

• the first-party localgpt-docs-theme cookie.

The cookie uses Path=/, SameSite=Lax, and a one-year lifetime. HTTPS pages add the Secure attribute. A tiny bootstrap script applies the stored value in the document head before the body is painted, reducing light/dark flicker.

The GitHub Pages origin and the local application origin have separate browser storage. A selection persists on each host, but browsers correctly do not share it across those origins.

Kawaii shell

The Kawaii layer is deliberately separate from DocFX's generated content. It provides the gradient shell, paw/cat branding, cards, subtle motion, cursor trail on fine pointers, and light/dark palettes. Reduced-motion and coarse-pointer users keep normal cursor and animation behavior.

PDF

The owner-side documentation build can assemble a complete PDF from generated conceptual and API HTML pages. It requires meaningful source-page and API-page counts and rejects a tiny fallback shell. Website-only motion and navigation are removed in print styles.

A prebuilt source package may include a curated handbook: all maintained conceptual chapters are printed in full, followed by a linked inventory of every generated API page. The full API reference remains available in HTML, and the regular LocalGPT build can regenerate the complete HTML-backed PDF when .NET, DocFX, and a supported browser are present.

Publishing verification

The Pages extractor verifies:

- index.html and api/index.html;
- current Kawaii CSS and JavaScript markers;
- documentation-status.json (including UTF-8 BOM compatibility);
- project-relative assets;
- a nonzero API count;
- the versioned PDF when available.

The Node deprecation messages emitted by GitHub's own actions are warnings; deployment success is determined by artifact upload and the Pages deployment result.

guide/embedded-and-games.md

Embedded planning, GameDirector, and creative runtimes

Embedded firmware planning

LocalGPT can turn a board description, pin layout, sensor/actuator roles, transport requirements, and policy constraints into a reviewable firmware plan.

The planner is transport-neutral. GPIO, ADC, PWM, I²C, SPI, UART, CAN, RS-485, physical 1-Wire, and LocalGPT logical telemetry are capabilities—not assumptions.

A plan can include:

• board and pin assignments;

• electrical or protocol findings;

• firmware module layout;

• telemetry contract;

• generated source artifacts;

• compiler and flashing prerequisites.

Planning, compilation, serial access, flashing, and actuator control are distinct operations. The later stages require the matching workspace and approval checks.

Organic 1-Wire

LocalGPT's “organic 1-Wire” is an application protocol for approved peers, publishers, add-ons, and devices. It is not limited to the Dallas/Maxim physical bus. Transport adapters can use TCP, HTTP/JSON, or a gateway while the application contract keeps identity, replay protection, approval, and capability routing explicit.

GameDirector

GameDirector is the authoritative game engine. Player controllers, creature Councils, and reactive map objects submit proposals; they do not mutate world state directly.

The deterministic resolver validates legality, turn order, movement, and state transitions. A configured model can enrich narration or predict consequences without owning the authoritative frame.

Runtime classes describe sessions, maps, players, directors, creatures, objects, and frames. Council roles can be assigned to world actors while the actual transition remains inside GameDirector.

ASCII play surface

The Chat ASCII console presents a stable 80×25 authoritative frame with responsive Fit, Width, and Native modes. It can be mounted or closed without deleting the underlying session. Fullscreen exit occurs before the component is removed so the normal conversation layout is restored correctly.

Minecraft Mod AI Builder

The Minecraft builder applies the same project principles: source knowledge is versioned, generated artifacts are reviewable, dependencies are explicit, and a generated mod/datapack is not presented as tested until the correct toolchain has actually built or validated it.

Historical source comparisons and early game presets are retained as internal notes, while this page describes the maintained runtime contract.

guide/getting-started.md

Getting started

Application shape

LocalGPT is one local application with several cooperating surfaces:

- Chat for ordinary provider-backed conversation.
- AI Council for role-based multi-model work.
- Projects and Project Maintenance for durable requirements, revisions, artifacts, workspaces, and toolchains.
- SQLite Database for inspected local data and persistent configuration.
- DX Functions & 1-Wire for bounded callable functions and peer/device operations.
- Embedded workbench, Minecraft Mod Builder, Test Lab, and other focused tools.
- Help for the generated DocFX site and API reference.

The desktop wrapper and web host expose the same backend application. The wrapper is a host, not a second architecture.

Configure a model provider

Provider identity is more than a model name. LocalGPT addresses a model using:

```
provider kind + provider name + endpoint + provider-native model name
```

This prevents an Ollama model and an LM Studio/OpenAI-compatible model with the same visible name from being merged accidentally.

A practical first setup is:

1. Configure or discover one local provider.
2. Test its endpoint.
3. Select one discovered model for Chat.
4. Send a harmless request before enabling Council or tool workflows.
5. Add credentials only to the exact endpoint that requires them.

Credentials are not copied to fallback endpoints and are not displayed in documentation or diagnostics.

Understand local data

LocalGPT stores durable application state in SQLite through EF Core. Depending on the enabled features, this includes projects, revisions, requirements, Council configuration, reviewed knowledge, runtime policies, presets, collaboration requests, and bounded operational records.

The database is not an authority source by itself. A stored row can describe a prior approval or preference, but it cannot manufacture a new approval for a consequential action.

Read-only work versus execution

LocalGPT separates four stages:

1. Inspect — read bounded files, metadata, provider catalogs, logs, or project state.

2. Plan — create a reviewable proposal, command plan, firmware plan, or change set.

3. Approve — collect the required user decision for the exact target and scope.

4. Execute — perform the bounded operation and record a sanitized result.

A failure at any stage remains visible. The application should report the missing dependency, source, permission, compiler, provider, or capability rather than claiming success.

Theme and documentation preference

The documentation theme selector supports Light, Dark, and Auto. The selection is stored in both browser storage and a first-party cookie named localgpt-docs-theme, so it persists when revisiting the GitHub Pages site or the documentation hosted by the local application. The two hosts keep their own preference because browsers isolate storage by origin.

Next step

Continue with Chat and AI Council (chat-and-council.md), or jump to Projects and workspaces (projects-and-workspaces.md) when your goal is code, documents, firmware, or another maintained artifact.

guide/index.md

User guide

Welcome to the practical side of LocalGPT. The interface is playful; the operational boundaries are not. A tiny bit kawaii, a lot of explicit control. ■

Start here

1. Getting started (getting-started.md) — application shape, providers, storage, and approval boundaries.

2. Chat and AI Council (chat-and-council.md) — direct conversations, teams, roles, provider-qualified models, benchmarks, and human participation.

3. Projects and workspaces (projects-and-workspaces.md) — durable projects, revisions, tracked files, toolchains, and permission assessment.

4. Embedded planning and games (embedded-and-games.md) — firmware plans, logical telemetry, GameDirector, ASCII play surfaces, and Minecraft generation.

5. Documentation inside LocalGPT (documentation.md) — HTML, API reference, PDF, GitHub Pages, search, and theme persistence.

What LocalGPT does not do silently

LocalGPT does not treat an AI answer as authorization. Consequential actions—writes, builds, native commands, serial access, flashing, actuator control, or protected peer operations—must cross an explicit application policy and, where required, a fresh user approval.

The application may prepare plans, drafts, diffs, firmware layouts, Council proposals, and capability reports without pretending those artifacts have already been executed.

> [!TIP]

> Begin with one provider and one project. Add Council members, runtime classes, and toolchains after the basic route works. Cute complexity is still complexity.

guide/projects-and-workspaces.md

Projects and workspaces

Durable project model

A LocalGPT project is language-neutral. It can describe a .NET solution, firmware workspace, game content, documentation set, or another technical/creative structure.

The durable model separates:

• project identity and purpose;

• versions and revisions;

• requirements and requirement links;

• named artifacts;

• topics and reviewed knowledge;

• tracked files and file-pattern rules;

• workspace roots and environment profiles;

• compiler installations and build evidence.

A filesystem path helps locate a checkout; it is not the identity of the project. Stable database identifiers survive moves and alternate workspaces.

Revisions and tracked files

A revision records ancestry, status, structure metadata, and build evidence. A tracked file uses a normalized relative path and stable identity. Hashes provide evidence that the file being reviewed is the file that was assessed; they do not grant permission to overwrite it.

The typical flow is:

1. register or select a project;

2. resolve an allowed workspace root;

3. scan expected files using maintained patterns;

4. create or select a revision;

5. inspect and propose changes;

6. save a review record;

7. approve the exact revision for testing;

8. run bounded verification through the configured toolchain.

Workspace permission assessment

A workspace contains rules for allowed paths, expected structure, compiler profiles, and environment variables. Findings are grouped as:

• Approved — the required operation is inside the allowed boundary;

• Warning — the operation may proceed only after visible review;

• Danger — the operation is blocked until the boundary is corrected.

Compiler execution requires a fresh assessment with proven access and no unresolved danger finding. A prior successful build does not automatically approve a later command against changed files or another path.

Toolchains

Compiler installations are records, not guessed commands. A toolchain can describe executable path, version, environment, supported project types, and validation state. Native execution goes through the command runner and approval policy rather than direct cmd.exe composition.

Collaboration and knowledge

Projects can link reviewed Council knowledge to topics and versions. Knowledge entries keep provenance, review state, archival state, and optional expiration. Automatic context should use only entries that remain current and explicitly eligible.

Artifacts

Generated source, documents, firmware, reports, and other outputs belong to reviewable artifact workspaces. Saving an artifact is separate from compiling, flashing, publishing, or replacing a project file.

Continue with Project and data architecture (../architecture/project-data.md) for the service and persistence boundaries.

architecture/ai-host.md

AI Host control plane

The AI Host control plane is a first-class part of LocalGPT's architecture. It defines how providers, model catalogs, runners, downloads, settings, logs, chat, and API-compatible routes fit behind explicit .NET service boundaries.

Purpose

The AI Host boundary gives LocalGPT a provider-neutral way to discover, configure, run, and observe compatible model runtimes while keeping native inference engines replaceable.

It separates four concerns:

1. Control plane — model catalog, running-model state, settings, downloads, health, logs, and request validation.
2. Provider adapter — Ollama, OpenAI-compatible, OpenAI, Azure OpenAI, or another bounded route.
3. Native/model runtime — the process or library that owns inference, model loading, and hardware use.
4. LocalGPT orchestration — Chat/Council selection, benchmarks, presets, policy, and UI.

LocalGPT should not pretend that rebuilding a high-performance inference engine in managed code is a small UI task. A .NET host can provide a strong control plane, provider-compatible API, native runner boundary, and plugin model while delegating optimized inference to a suitable runtime.

Provider-qualified identity

A model route contains:

• provider kind;
• configured provider name;
• endpoint;
• provider-native model name;
• optional credentials held by the configured provider;
• capabilities and provider-specific options.

The route is the identity. Model-name strings alone are insufficient.

Core services

A practical .NET architecture uses contracts similar to:

Service	Responsibility
IProviderModelRuntimeService	Discover and invoke provider-qualified routes
IChatClientFactory	Create the correct client for a route immediately before use
IOllamaProcessService	Inspect or control a bounded local Ollama process when explicitly configured
IAiConnectivityProbe	Test provider health without starting unrelated processes
IModelPresetService	Store LocalGPT routing presets

| IProviderModelBenchmarkService | Run bounded benchmark tasks and recommendations |

| INativeCommandRunner | Execute approved native commands without shell-string composition |

A host-specific adapter owns route translation, streaming payloads, error normalization, and capability discovery. The Council layer consumes the common route model rather than branching on provider details in Razor components.

API families

A compatible control plane commonly needs route families for:

• health and version;

• model catalog and model details;

• installed/running models;

• chat/generate/embedding requests;

• streaming responses and cancellation;

• pull/download progress;

• load/unload/keep-alive;

• runtime settings and resource limits;

• logs and diagnostics.

Compatibility must be explicit. A route should not be advertised until its request, response, streaming, cancellation, and error behavior are implemented and tested.

Native runner boundary

The managed application owns configuration and policy; a native runner owns the executable process. The boundary should provide:

• executable and argument arrays rather than shell command strings;

• working-directory and environment allowlists;

• cancellation and timeout;

• bounded stdout/stderr capture;

• process identity and exit code;

• no automatic elevation;

• no execution of uploaded binaries.

Plugins or runners should be registered from reviewed application configuration, not discovered and executed from arbitrary archives.

Model lifecycle

Model acquisition is a separate workflow from model use. A safe lifecycle includes:

1. user selects a provider and model source;

2. LocalGPT shows size, destination, compatibility, and checksum/signature information when available;

3. the user approves the download;

4. progress is recorded without leaking credentials;

5. the adapter verifies the result;

6. the model enters the catalog;

7. load/unload remains explicit and observable.

A benchmark may recommend context/output bounds or an Ollama GPU profile. It does not pull a model or rewrite provider-global settings automatically.

Blazor and DevExpress surfaces

A useful host UI can include:

• provider dashboard and health cards;

• model catalog and installed-model grid;

• active model/process view;

• chat playground;

• benchmark panel;

• download queue;

• settings and diagnostics;

• API compatibility status.

DevExpress grids, forms, dialogs, tabs, and charts should be used where they add concrete behavior. Bootstrap owns the responsive shell. Pages call application services; they do not become provider adapters.

Multi-model operation

LocalGPT may run several compatible routes at once when hardware, provider behavior, and policy allow it. Resource planning should remain visible: model size, estimated memory, context, output bounds, concurrency, and provider limits are inputs to the plan.

The architecture must degrade honestly. If a requested model cannot be loaded beside another one, the host reports the constraint and offers a reviewable unload/retry plan instead of silently replacing the active model.

Acceptance target

A LocalGPT AI Host milestone is complete only when:

• provider-qualified routes remain distinct;

• health/discovery and invocation use the same configured endpoint;

• streaming and cancellation work;

• native execution is bounded;

• credentials stay scoped;

• model lifecycle actions are explicit;

• the UI shows real state;

• benchmark recommendations require user application;

• compatibility claims are backed by tests.

architecture/council-runtime.md

Chat and Council runtime

Session boundary

A chat session combines a selected provider route, protocol profile, formatter, prompt configuration, function catalog, and bounded context. These dependencies are resolved through services and remain stable for the response stream.

Protocol profiles are stateless. They detect provider/model families and normalize only their own control markers. The response formatter owns presentation of thinking, final content, tools, code, and structured sections.

Council orchestration

A Council request is decomposed into explicit workflow steps. The runtime owns:

- team and member configuration;
- provider-qualified route resolution;
- preflight and readiness checks;
- role/task prompts;
- step ordering and cancellation;
- human questions and decisions;
- final-result composition;
- durable summaries and artifacts where appropriate.

The spooler represents queued/running work. Runtime classes describe reusable behavior or game actors, but they do not bypass the workflow service.

Deterministic completion

Models are probabilistic; step completion is not. The application evaluates whether a response contains substantive final content, whether required outputs exist, and whether a bounded recovery is allowed.

A verifier can add confidence or a warning. It cannot retroactively make an invalid primary result valid. Conversely, a failed verifier should not erase a valid result when the workflow contract treats verification as advisory.

Human collaboration

Human participation is modeled as a request and decision flow with an exact question, boundary, status, and optional reuse scope. It supports:

- answering a Council question;
- choosing among alternatives;
- reviewing a proposed change;
- approving a bounded operation;
- contributing domain knowledge.

A human decision is bound to its context. It is not a global bypass token.

DX functions

DX functions expose typed application operations to chat and Council workflows. The catalog describes parameters and capabilities. The handler/service still owns validation, policy, and execution.

Functions should return structured outcomes that distinguish:

- success;
- user decision required;
- capability missing;
- validation failure;
- execution failure;
- partial/reviewable output.

Change reviews

Generated changes are stored as reviewable records rather than immediately applied. A review can include proposed files, diffs or replacement content, architecture findings, build instructions, execution evidence, and user decisions.

The workflow can approve a revision for testing without marking it released or deployed.

Knowledge

Council knowledge is persistent but curated. Automatic context uses reviewed, active, non-expired entries whose provenance is acceptable. Source-backed knowledge is refreshed when its source hash changes. Raw uploads and model answers do not become trusted knowledge automatically.

GameDirector reuse

GameDirector uses the same Council machinery for role-assigned world actors while retaining a deterministic authoritative resolver. This is an example of the intended architecture: generative proposals inside a strict application-owned state machine.

architecture/frontend-and-themes.md

Frontend, DevExpress, and themes

UI ownership

Bootstrap v5 owns responsive layout, spacing, breakpoints, and the application shell. DevExpress components provide behavior-rich grids, editors, dialogs, charts, tabs, menus, and chat surfaces where their features are actually needed.

Razor components present state and call services. They do not own provider protocols, command execution, database initialization, or file policy.

Component contract

Maintained components use explicit injected dependencies for logging, notification, and bounded activity reporting. Long operations support cancellation and show progress. Dialogs have one internal scroll owner and remain usable at ordinary browser zoom.

A failure boundary prevents one component exception from taking down the full circuit when a recoverable user-facing result is possible.

Design patterns

• persistent navigation shell;

• responsive card/grid layouts;

• collapsible advanced configuration;

• reusable provider-model panel;

• status chips with text, not color alone;

• confirmation dialogs that name the exact target;

• bounded tables with search/filter;

• empty states that explain the next action;

• no fake controls or decorative buttons without behavior.

DevExpress assets

DevExpress packages and runtime assets remain governed by their own license. Generated license material, proprietary distribution assets, or credentials must not be committed accidentally. Build and publish scripts use the configured package source and fail clearly when licensed dependencies are unavailable.

Theme Fusion in the application

The application theme system keeps two selections:

1. Base Theme — shell/background, Bootstrap metadata, syntax highlighting, and native LocalGPT surfaces.

2. Style Layer — optional blend/override for DevExpress and other component surfaces.

Selections are persistent and applied through a service/JavaScript boundary. A theme failure falls back to a readable base rather than leaving the UI half-styled.

Runtime theme ownership

ThemeService is scoped and keeps independent Base Theme and Style Layer selections. MenuIsland owns the interactive render mode; nested switcher components inherit that circuit. Startup registers the active component theme through DxResourceManager.RegisterTheme, while runtime component-theme changes await IThemeChangeService.SetTheme(...). Base-only changes do not inject a competing DevExpress stylesheet.

The browser boundary persists validated names, sets data-bs-theme and LocalGPT theme metadata, and optionally updates syntax highlighting. A failed change restores the prior readable state. The maintained catalog still supports classic themes such as Blazing Berry alongside newer palettes.

Documentation theme

The DocFX site uses a separate Kawaii shell around generated content. It supports light, dark, and auto modes through the standard DocFX selector, with LocalGPT persistence layered on top.

The theme script:

- applies preference before paint;
- stores it in local storage and a first-party cookie;
- observes DocFX changes without replacing its dropdown;
- keeps the dropdown above decorative layers;
- respects reduced-motion and coarse-pointer preferences;
- decorates the brand with a real paw/cat SVG and favicon.

JavaScript boundary

JavaScript is used for browser-only behavior: theme metadata, focus/layout helpers, fullscreen, file download, documentation decoration, and DevExpress interop. Each entry point is registered and diagnostically traceable. Missing functions fail with a useful message rather than a silent undefined call.

Testing

Frontend validation combines static checks, component tests where available, browser automation for important routes, and manual visual review at responsive sizes. A screenshot alone is not a behavioral test, but it is useful evidence for layout regressions.

architecture/index.md

Architecture

LocalGPT is a modular monolith: one deployable local application with explicit service boundaries, provider adapters, persistence, Blazor UI, and optional host wrappers.

Read in this order

1. System overview (system-overview.md)
2. AI Host control plane (ai-host.md)
3. Chat and Council runtime (council-runtime.md)
4. Projects, workspaces, and persistence (project-data.md)
5. Organic 1-Wire and security (onewire-security.md)
6. Frontend, DevExpress, and themes (frontend-and-themes.md)

Dependency direction

```
flowchart TD
    UI[Components and controllers] --> SVC[Application services]
    SVC --> CONTRACTS[Interfaces and business contracts]
    SVC --> ADAPTERS[Provider / transport / tool adapters]
    ADAPTERS --> EXT[External runtimes and devices]
    SVC --> DATA[EF Core persistence]
    DATA --> DB[(SQLite)]
```

Components coordinate user interaction. Services own operations. Adapters own protocol details. Persistence owns durable state. Static helpers may format or parse immutable data, but application-owned mutable state belongs to scoped, singleton, or hosted services with explicit lifetimes.

> [!IMPORTANT]

> Architecture documents describe boundaries; they do not authorize execution. The current user interaction remains the authority source.

architecture/onewire-security.md

Organic 1-Wire and security

Protocol meaning

LocalGPT's organic 1-Wire is a transport-neutral application protocol for cooperation between LocalGPT, PublisherStudio-style peers, approved add-ons, and device gateways. It may travel over TCP, HTTP/JSON, or another bounded transport. It is not the physical Dallas/Maxim 1-Wire protocol, though a gateway may bridge to physical hardware.

Identity

Each application creates its own local identity material at runtime. Private keys, shared secrets, certificates, MFA seeds, and trusted-peer databases are not committed to source or embedded in releases.

Identity creation, regeneration, deletion, and peer trust are visible user operations. Read-only program directories fall back to the user's application-data location.

Connection flow

A safe peer flow is:

1. discover a peer without auto-trusting it;
2. user selects the peer;
3. exchange and display identity information;
4. user confirms the intended relationship;
5. establish the protected session;
6. advertise bounded capabilities;
7. submit work through the spooler;
8. require fresh approval for protected operations.

A sender-supplied UserConfirmed=true flag is data, not proof of local approval.

Envelope and replay protection

Protected messages use an envelope with stable identifiers, timestamps/nonces, target capability, payload, and authentication material. The receiver validates identity, freshness, replay policy, capability, and local approval before dispatch.

Replay records are durable where needed and bounded by retention policy. Invalid envelopes fail without exposing secrets or executing partial work.

Capability routing

The capability catalog maps protocol operations to application services. It prevents transport handlers from reflecting arbitrary public methods or constructing unreviewed commands.

Capabilities may include reviewed text exchange, screen capture requests, website/document content, embedded wiring proposals, OCR, or plugin work results. Each capability has its own parameters and approval behavior.

Work spooler

Incoming work enters a queue with peer identity, capability, payload summary, status, and decision state. A hosted processor executes only work that has passed validation and approval.

Security invariants

• no trust on discovery alone;

• no credential reuse across unrelated endpoints;

• no arbitrary reflection execution;

• no uploaded executable execution;

• no automatic filesystem write outside approved workspace;

• no actuator/flash/native command without the matching approval;

• no secret content in logs or documentation;

• no model response treated as authority.

The protocol remains useful because the boundaries are explicit, not because every peer is assumed friendly. ■

architecture/project-data.md

Projects, workspaces, and persistence

Database-first project model

LocalGPT stores the project model in SQLite through EF Core. The database describes the user's technical intent independently of a particular checkout path or language.

Core records include projects, versions, revisions, requirements, requirement links, artifacts, topics, knowledge links, tracked files, workspace roots, compiler installations, build verifications, and Council review records.

Identity and paths

Stable identifiers belong to database records. Paths are mutable location data. A project can have several workspace roots or move to another machine without changing its conceptual identity.

Tracked files use normalized relative paths and hashes. This supports exact-source review and prevents a report about one file version from being mistaken for evidence about another.

Workspace service boundary

The workspace resolver maps a project/revision to an approved root and validates:

• path containment;

• expected structure;

• read/write requirements;

• file-pattern policy;

• environment variables;

• compiler/tool availability;

• danger findings.

The project service does not execute compilers. The artifact service does not silently replace tracked files. The command runner does not decide project policy.

Compiler inventory

Compiler installations are explicit records with path, version, supported workload, and validation state. Discovery can propose installations; the user selects what the project may use.

Build verification stores the command plan, revision, timing, exit status, bounded output, and artifact evidence. A successful historical record is not a standing approval for future execution.

EF Core initialization

Database initialization supports both clean installs and older development databases that may contain application tables before migration history was complete.

The bootstrap sequence must:

1. open the configured database safely;

2. inspect migration history and existing schema;

3. distinguish an empty database from a legacy schema;

4. apply or baseline migrations in the maintained order;

5. preserve user data;

6. initialize seed/catalog data idempotently;

7. expose readiness to dependent services.

Logging tables are handled carefully so diagnostic logging does not recurse into an unready database.

Migration snapshots

EF migration snapshots and generated migrations are architecture artifacts. They must remain ordered, compilable, and consistent with the context model. Manual fixes should not create duplicate identities or silently drop columns.

Knowledge and runtime policy

Seed services initialize known catalogs and policy definitions. Mutable values are stored in the database or owning service, not in global static collections. User changes remain distinct from shipped defaults.

Durable entity contract

The database-first model keeps the important records explicit:

• LocalGptProject owns revisions, requirements, artifacts, topics, and versions.

• LocalGptProjectRevision stores ancestry, status, structure metadata, and build evidence without touching Git or files by itself.

• LocalGptProjectRequirementLink maps a requirement to a stable function, service, controller, business object, table, configuration, variable, regex, prompt, knowledge entry, or generated-code target.

• ProjectDocumentImport stores bounded normalized text with source, hash, and safety metadata.

Before a Council step calls a function, it identifies the project/revision, maps the task to approved requirements, states missing evidence, and chooses the smallest relevant function set. Function availability is not a reason to call it.

Migration compatibility sequence

Older owner databases can contain application tables before EF migration history was complete. Compatibility handling therefore performs SQLite health checks, schema inspection, stale-lock handling, and a SQLite online backup before adopting history or migrating a compatible logging table. A history row is inserted only when the migration's complete signature is present. Refuse partially applied ambiguous schemas and report the missing markers plus the backup path instead of guessing.

The database logger uses a separate readiness gate so queued startup diagnostics cannot write through EF while migrations and deterministic seed stages are still running.

Snapshot ordering contract

LocalGptMemoryDbContextModelSnapshot is executable model-building code. Keep properties first, configure relationships only after both entity property blocks exist, and add collection navigations after the matching relationships. An early navigation-only call can accidentally create a shared Dictionary<string, object> entity under the CLR type name.

Run build/Assert-EfSnapshotArchitecture.ps1 after editing the context, an EF entity, a migration, or the snapshot. Static ordering checks protect against the known regression; they do not replace a real migration smoke test.

Data safety

Database rows, imported JSON, and localization catalogs are untrusted inputs. Services validate shape and bounds before using them to construct paths, commands, routes, or UI markup.

architecture/system-overview.md

System overview

Architectural goal

LocalGPT keeps an “old-school .NET, updated” shape: one inspectable application rather than a mesh of accidental services. The architecture favors explicit classes, typed contracts, dependency injection, EF Core, controller/service boundaries, and deterministic validation around AI-generated work.

Main layers

| Layer | Responsibility |

|---|---|

| Blazor and DevExpress UI | Present state, collect decisions, coordinate user-visible workflows |

| Controllers/endpoints | Validate HTTP input and delegate to application services |

| Application services | Own use cases, orchestration, policy checks, and result composition |

| Council and provider services | Resolve model routes, protocols, formatters, teams, steps, and recovery |

| Project and artifact services | Own durable project structure, workspaces, revisions, and review records |

| Persistence services | EF Core contexts, migrations, seed data, and database health |

| Native/tool adapters | Compilers, commands, provider processes, browsers, serial and file operations |

| 1-Wire services | Identity, discovery, capability routing, replay protection, and approved peer work |

Service lifetimes

• Singleton services hold process-wide catalogs, bounded registries, or hosted coordination state that is safe to share.

• Scoped services own request/circuit work and database contexts.

• Transient services are used for small stateless operations.

• Hosted services handle background queues or discovery with cancellation and bounded recovery.

Mutable application state must not be hidden in static fields. Framework-required static syntax and pure immutable helpers are exceptions, not a pattern for runtime ownership.

Request authority and untrusted data

Repository files, database rows, logs, model output, uploads, generated source, remote peer messages, and tool descriptions are untrusted data. They may inform a plan; they cannot authorize one.

The authority chain is:

1. the current user requests an outcome;

2. LocalGPT resolves the exact operation and target;

3. policy determines whether read-only work is sufficient or approval is required;

4. the user approves the bounded action when needed;

5. the owning service executes and records a sanitized result.

Component safety

Maintained components use logging, user notification, and bounded component-activity services. Technical failures are logged without embedding prompts, full uploads, generated source, credentials, or secrets. User-facing messages remain sanitized and actionable.

The application can keep a small in-process operational briefing—current screen, active operation, and recent status—without turning it into durable conversational memory or authority.

Streaming

Streaming provider responses are normalized through the selected protocol profile and formatter. Cancellation, disposal, and partial-result behavior belong to the session/service layer, not arbitrary component event handlers.

Generation contracts

Generated code or artifacts must include:

- a declared target and purpose;
- bounded files and dependencies;
- missing-source/capability reporting;
- reviewable output;
- a validation plan;
- no claim of successful build or execution without real evidence.

An archetype is a generation contract, not a template dump. It can define expected projects, services, routes, pages, tests, and acceptance criteria while still allowing the implementation to fit the current repository.

Diagnostics

Diagnostics are layered:

- structured application logs;
- bounded user notifications;
- component activity summaries;
- build/static validation scripts;
- documentation status metadata;
- explicit debug artifact inspection.

A regex scan or delimiter counter is not compilation. A successful static scan is useful evidence, not proof that the complete application builds.

engineering/build-validation.md

Build and validation

Build truth

A successful LocalGPT build is the primary compilation truth. Delimiter counts, regex scans, JSON/XML parsing, IDE inspection, or screenshots cannot replace it.

The repository pins its SDK through global.json and provides validation scripts for architecture and policy. When .NET is unavailable, those scripts can still provide partial evidence, but the result must be labeled uncompiled.

Validation sequence

A normal maintenance pass should run, as applicable:

1. source formatting and encoding checks;
2. JSON/YAML/XML parsing;
3. C# syntax parsing through the pinned SDK/Roslyn route;
4. architecture assertions;
5. dependency and publish-configuration checks;
6. localization and JavaScript diagnostics checks;
7. database migration/snapshot checks;
8. application build/tests;
9. documentation build and artifact verification;
10. focused browser smoke tests.

Generated C#

Generated source must preserve namespaces, nullable context, project references, service lifetimes, and existing framework patterns. It must not report compilation success until the target project has actually compiled.

Warnings are triaged by category. Suppression is not the default response to a warning introduced by generated code.

Static and runtime ownership

Application-owned mutable catalogs, regex lists, runtime values, and policy state are not static. Framework bootstrap and pure immutable helpers remain valid static boundaries.

Runtime-value services own mutable definitions and current values. Persistence services own durable policy. Components consume services rather than constructing global state.

Logging integrity

Logs contain technical context needed for diagnosis but avoid prompts, full model responses, uploads, generated file contents, credentials, secrets, and unbounded exception data.

The database logger exposes readiness so startup failures do not recurse. User notifications are sanitized; detailed traces remain in the configured logger.

JavaScript diagnostics

Maintained JavaScript functions used through interop are registered and validated. Build checks detect missing function names and known unsafe patterns. Browser tests cover theme selection, documentation dropdown behavior, fullscreen/close flows, and critical navigation.

Frontend checks

Important checks include:

- interactive render-mode ownership;
- no nested incompatible render boundaries;
- responsive dialogs and scroll ownership;
- keyboard focus and visible labels;
- clickable menus above decorative layers;
- reduced-motion behavior;
- light/dark contrast;
- no fake or dead controls.

Documentation checks

The documentation build verifies conceptual pages, generated API YAML/HTML, versioned PDF, Kawaii CSS/JS markers, icon assets, status JSON, and nonzero source/API counts.

The Pages extractor accepts UTF-8 files with or without BOM, normalizes ZIP separators, rejects unsafe paths, and chooses the strongest complete release candidate.

engineering/index.md

Engineering guidance

This section collects the rules that keep LocalGPT maintainable after the architecture has become interesting enough to bite back.

- Build and validation (build-validation.md)
- Release and documentation pipeline (release-and-docs.md)
- Maintenance status (maintenance-status.md)

Working principle

Use the strongest available evidence and label weaker evidence honestly:

```
real build/test > parser/static validation > focused inspection > assumption
```

A capability gap should report the requested outcome, missing dependency or source, evidence inspected, safe next step, and whether owner-side tooling or approval is required.

Open work

Current owner-side verification tasks live on the maintenance status (maintenance-status.md) page. Historical task lists and pass manifests remain under docs/internal-notes and are excluded from the reader site.

engineering/maintenance-status.md

Maintenance status

This is the small, honest maintenance corner of the documentation. ■ It keeps owner-only validation and unfinished integration work visible without mixing temporary pass notes into the architecture chapters.

A task is complete only after implementation, compatibility review, validation, and user-visible verification. Drafted code or a successful static scan is useful evidence, but not the finish line.

Build and runtime gates

• Compile the root LocalGPT project and run startup against a backed-up owner database using the exact release candidate.

• Complete a licensed Windows/DevExpress Release build and the supported package matrix.

• Run real-package DevExpress and SQLite integration tests.

• Test supervised routes, collaboration refresh, and Chat autosave during rapid navigation, reconnect, and disposal.

• Review singleton registrations for thread safety; move circuit- or user-mutable state to scoped services where required.

Persistence and recovery gates

• Verify compatibility-backup creation, migration-history adoption, pending migrations, and initial catalog seeding.

• Inspect and record __EFMigrationsHistory after a successful owner run.

• Run migration and recovery smoke tests for empty, logging-only, untracked-current, partial, locked, and corrupt SQLite databases.

UI and theme gates

• Test runtime theme initialization, Light/Dark/Auto switching, rollback, reconnect, and disposal in the app and in the shipped DocFX site.

• Confirm the documentation theme preference persists independently on the local application origin and the GitHub Pages origin.

• Decide whether Highlight.js theme stylesheets should be vendored for completely offline switching.

• Complete rich SQLite custom mask, format, and null-text editing.

• Add a catalog-assisted requirement-link picker with validation.

Remaining integration work

• Continue converting legacy logger helpers to injected logging and bounded service activity at high-level boundaries.

• Complete repository-pull UI integration through safe text ingestion.

Carry-forward rule

Every unresolved item moves into the next current changelog, issue, or release-planning source. Historical pass manifests and superseded task lists remain available under docs/internal-notes, excluded from DocFX, so the public documentation stays focused without losing the trail that led here.

engineering/release-and-docs.md

Release and documentation pipeline

Release discipline

Releases use a clean, reviewable lane. The version must be greater than prior public releases, repository state must be understood, and required build/validation steps must finish before assets are published.

The release package matrix includes desktop/WebView and backend packages for supported Windows, Linux, macOS, and architecture targets. Setup packages remain separate from portable packages.

Documentation build

The build restores the repository-local DocFX tool, extracts metadata from LocalGPT.dll and LocalGPT.xml, builds conceptual and API pages, installs the Kawaii assets, produces the versioned PDF, writes documentation-status.json, and copies the complete tree into wwwroot/help-docs.

The build script injects:

- the localgpt-kawaii-docs HTML class;
- cache-busted Kawaii CSS/JS links;
- an early theme bootstrap;
- paw/cat favicon and brand assets.

GitHub Pages

The Pages workflow resolves a release tag, downloads release ZIPs, inspects each ZIP safely, and scores complete documentation candidates. It validates theme markers, hashes, index/API pages, status metadata, relative assets, and page counts before uploading one Pages artifact.

Windows ZIP backslashes and UTF-8 BOMs are accepted. Setup archives that do not contain the full site are ignored.

Node warnings

GitHub's official Pages and artifact actions may emit Node or dependency deprecation warnings. These warnings do not mean the static site runs Node. The deployed site is HTML, CSS, browser JavaScript, and assets. The workflow result and Pages deployment status determine success.

PDF

The preferred PDF route assembles the generated HTML pages and prints them with an installed Edge/Chrome/Chromium browser. A secondary DocFX PDF route may require Node. Both routes must produce a real, sufficiently sized document; a one-page fallback shell is rejected.

Stop conditions

Stop and report clearly when:

- the working tree or version is unsuitable;

- required licensed dependencies are unavailable;
- compilation or validation fails;
- the API graph is empty;
- the PDF is incomplete;
- release assets do not contain the complete docs tree;
- publishing credentials/permissions are unavailable.

reference/capability-map.md

Capability map

| Area | Maintained capability | Important boundary |

|---|---|---|

| Chat | Provider-qualified single-model sessions | Protocol/formatter isolation |

| AI Council | Teams, roles, runtime classes, workflows, human participation | Deterministic step state and bounded recovery |

| Providers | Ollama, OpenAI-compatible, OpenAI, Azure OpenAI routes | Endpoint-qualified identity and scoped credentials |

| Benchmarking | Bounded tasks, peer/self review, presets | Recommendations require user application |

| Projects | Versions, revisions, requirements, artifacts, topics | Database identity is separate from paths |

| Workspaces | Root resolution, policies, toolchains, build evidence | Fresh assessment before execution |

| Knowledge | Reviewed persistent entries and source refresh | Raw uploads/model output are not trusted automatically |

| DX functions | Typed callable application operations | Handler/service owns policy and execution |

| 1-Wire | Peer discovery, identity, capabilities, work spooler | Discovery is not trust; remote flags are not approval |

| Embedded | Board/pin/transport plans and artifacts | Plan, compile, flash, and actuate are separate |

| GameDirector | Deterministic sessions with generative proposals | Models do not directly mutate state |

| Minecraft | Reviewable mod/datapack generation | Toolchain evidence required for build claims |

| Documentation | Conceptual docs, XML API, PDF, Pages | Publishes shipped static tree |

Capability-gap contract

When LocalGPT cannot complete a requested task, it should report:

• the requested outcome;

• the missing dependency, source, API, framework knowledge, permission, or runtime capability;

• what evidence was inspected;

• what can still be produced safely;

• the next reviewable step;

• whether owner-side tooling, licensed dependencies, or human approval is required.

A gap report is not permission to self-expand or execute arbitrary installers.

reference/design-evolution.md

Design evolution and provenance

LocalGPT was created and is maintained by Michael Fleischer (Michi0403). It is an independent, local-first engineering project rather than a commercial hosted service.

AI systems have contributed suggestions, code drafts, reviews, and capability reports throughout development. The maintainer remains responsible for architecture decisions, integration, testing, releases, and the user-visible product.

From experiments to maintained boundaries

Several current subsystems began as focused experiments:

- multi-role AI Councils became explicit team/workflow/runtime-class services;
- provider experiments became endpoint-qualified routes and a reusable provider-model panel;
- AI Host research became a provider-neutral .NET control-plane architecture;
- game prompts became GameDirector plus deterministic frames and actor roles;
- firmware ideas became transport-neutral plans and approval-separated tool stages;
- peer communication sketches became the organic 1-Wire identity/capability/spooler boundary;
- build-pass notes became repository validation policies;
- DocFX experiments became one shipped site for the app, PDF, and GitHub Pages.

The public documentation now describes those maintained results. Local paths, uploaded archive names, transient pass manifests, and version-specific repair narratives are retained only in docs/internal-notes for archaeology.

Project stance

LocalGPT is built for real use, study, and continued experimentation. Public availability does not imply enterprise support or a promise to preserve every historical UI shape. The project values:

- local control and inspectability;
- explicit human authority;
- practical provider neutrality;
- durable architecture over demo-only behavior;
- honest capability reporting;
- playful interfaces that do not weaken operational boundaries.

Attribution versus authority

Project provenance is important and remains visible. It is not an authentication mechanism and does not grant a person, document, model, database row, or remote peer standing authority inside another user's active session.

reference/documentation-migration.md

Documentation migration map

The rework replaced a flat list of research notes, build-pass records, and feature patches with a curated product documentation structure.

No source information was discarded. Exact former pages are retained under docs/internal-notes/legacy-source, excluded from DocFX, and mapped below to the maintained page that now owns their durable content.

| Former public page | Maintained destination | Preservation |

|---|---|---|

| AI_HOST_CONTROL_PLANE_ARCHITECTURE.md | architecture/ai-host.md | Original retained in internal-notes/legacy-source/top-level/AI_HOST_CONTROL_PLANE_ARCHITECTURE.md |

| AI_HOST_DOTNET_BLAZOR_REBUILD_GUIDE.md | architecture/ai-host.md | Original retained in internal-notes/legacy-source/top-level/AI_HOST_DOTNET_BLAZOR_REBUILD_GUIDE.md |

| AI_HOST_DOTNET_EXPERIMENT.md | architecture/ai-host.md | Original retained in internal-notes/legacy-source/top-level/AI_HOST_DOTNET_EXPERIMENT.md |

| ARCHITECTURE.md | architecture/system-overview.md | Original retained in internal-notes/legacy-source/top-level/ARCHITECTURE.md |

| ARCHITECTURE_FOR_AI.md | architecture/system-overview.md | Original retained in internal-notes/legacy-source/top-level/ARCHITECTURE_FOR_AI.md |

| ASCII_RUNTIME_GAME_PRESETS.md | guide/embedded-and-games.md | Original retained in internal-notes/legacy-source/top-level/ASCII_RUNTIME_GAME_PRESETS.md |

| BLAZOR_BOOTSTRAP_DEVEXPRESS_DESIGN.md | architecture/frontend-and-themes.md | Original retained in internal-notes/legacy-source/top-level/BLAZOR_BOOTSTRAP_DEVEXPRESS_DESIGN.md |

| BLAZOR_DEVEXPRESS_AI_GENERATION.md | architecture/frontend-and-themes.md | Original retained in internal-notes/legacy-source/top-level/BLAZOR_DEVEXPRESS_AI_GENERATION.md |

| CAPABILITY_GAP_CONTRACT.md | reference/capability-map.md | Original retained in internal-notes/legacy-source/top-level/CAPABILITY_GAP_CONTRACT.md |

| CHAT_PROTOCOLS_AND_SESSION_CONTEXT.md | architecture/council-runtime.md | Original retained in internal-notes/legacy-source/top-level/CHAT_PROTOCOLS_AND_SESSION_CONTEXT.md |

| COMPILER_VALIDATION_AND_GENERATION_RULES.md | engineering/build-validation.md | Original retained in internal-notes/legacy-source/top-level/COMPILER_VALIDATION_AND_GENERATION_RULES.md |

| COMPONENT_SAFETY_AND_SHORT_TERM_MEMORY.md | architecture/system-overview.md | Original retained in internal-notes/legacy-source/top-level/COMPONENT_SAFETY_AND_SHORT_TERM_MEMORY.md |

| DATABASE_FIRST_PROJECT_ARCHITECTURE.md | architecture/project-data.md | Original retained in internal-notes/legacy-source/top-level/DATABASE_FIRST_PROJECT_ARCHITECTURE.md |

| DATABASE_MIGRATION_BOOTSTRAP.md | architecture/project-data.md | Original retained in internal-notes/legacy-source/top-level/DATABASE_MIGRATION_BOOTSTRAP.md |

| DEVEXPRESS_ASSETS.md | architecture/frontend-and-themes.md | Original retained in internal-notes/legacy-source/top-level/DEVEXPRESS_ASSETS.md |

| DOTNET_AI_HOST_ARCHITECTURE_PATTERNS.md | architecture/ai-host.md | Original retained in internal-notes/legacy-source/top-level/DOTNET_AI_HOST_ARCHITECTURE_PATTERNS.md |

| DXAI_FUNCTIONS_AND_CHANGE_REVIEWS.md | architecture/council-runtime.md | Original retained in internal-notes/legacy-source/top-level/DXAI_FUNCTIONS_AND_CHANGE_REVIEWS.md |

| EF_DEVEXPRESS_BUSINESS_OBJECTS.md | architecture/project-data.md | Original retained in internal-notes/legacy-source/top-level/EF_DEVEXPRESS_BUSINESS_OBJECTS.md |

| EF_MIGRATION_SNAPSHOT_ARCHITECTURE.md | architecture/project-data.md | Original retained in internal-notes/legacy-source/top-level/EF_MIGRATION_SNAPSHOT_ARCHITECTURE.md |

| FRONTEND_DESIGN_PATTERN_LIBRARY.md | architecture/frontend-and-themes.md | Original retained in internal-notes/legacy-source/top-level/FRONTEND_DESIGN_PATTERN_LIBRARY.md |

| FRONTEND_TEST_AUTOMATION.md | engineering/build-validation.md | Original retained in internal-notes/legacy-source/top-level/FRONTEND_TEST_AUTOMATION.md |

| GENERATION_ARCHETYPE_CONTRACTS.md | architecture/system-overview.md | Original retained in internal-notes/legacy-source/top-level/GENERATION_ARCHETYPE_CONTRACTS.md |

| HUMAN_AI_COLLABORATION.md | architecture/council-runtime.md | Original retained in internal-notes/legacy-source/top-level/HUMAN_AI_COLLABORATION.md |

| JAVASCRIPT_RUNTIME_DIAGNOSTICS.md | engineering/build-validation.md | Original retained in internal-notes/legacy-source/top-level/JAVASCRIPT_RUNTIME_DIAGNOSTICS.md |

| LOCALGPT_CAPABILITY_SNAPSHOT.md | reference/capability-map.md | Original retained in internal-notes/legacy-source/top-level/LOCALGPT_CAPABILITY_SNAPSHOT.md |

| LOCALGPT_DEVELOPER_DIARY.md | reference/design-evolution.md | Original retained in internal-notes/legacy-source/top-level/LOCALGPT_DEVELOPER_DIARY.md |

| LOCALGPT_WORKFLOW_MEMORY.md | reference/design-evolution.md | Original retained in internal-notes/legacy-source/top-level/LOCALGPT_WORKFLOW_MEMORY.md |

| LOGGING_INTEGRITY.md | engineering/build-validation.md | Original retained in internal-notes/legacy-source/top-level/LOGGING_INTEGRITY.md |

| MICROSOFT_DOTNET_SAMPLE_CURRICULUM.md | architecture/frontend-and-themes.md | Original retained in internal-notes/legacy-source/top-level/MICROSOFT_DOTNET_SAMPLE_CURRICULUM.md |

| MINECRAFT_MOD_AI_BUILDER.md | guide/embedded-and-games.md | Original retained in internal-notes/legacy-source/top-level/MINECRAFT_MOD_AI_BUILDER.md |

| MINECRAFT_SOURCE_KNOWLEDGE.md | guide/embedded-and-games.md | Original retained in internal-notes/legacy-source/top-level/MINECRAFT_SOURCE_KNOWLEDGE.md |

| ONEWIRE_RUNTIME_SECURITY_HTTP_JSON.md | architecture/onewire-security.md | Original retained in internal-notes/legacy-source/top-level/ONEWIRE_RUNTIME_SECURITY_HTTP_JSON.md |

| OPEN_TASKS.md | engineering/maintenance-status.md | Original retained in internal-notes/legacy-source/top-level/OPEN_TASKS.md |

| OPEN_WIRINGS_2.0.3.md | reference/design-evolution.md | Original retained in internal-notes/legacy-source/top-level/OPEN_WIRINGS_2.0.3.md |

| ORGANIC-AI-COUNCIL-BLUEPRINT-v1.6.md | architecture/onewire-security.md | Original retained in internal-notes/legacy-source/top-level/ORGANIC-AI-COUNCIL-BLUEPRINT-v1.6.md |

| ORGANIC_AI_COUNCIL_BLUEPRINT_2_1.md | architecture/onewire-security.md | Original retained in internal-notes/legacy-source/top-level/ORGANIC_AI_COUNCIL_BLUEPRINT_2_1.md |

| PROJECT-MAINTENANCE-WORKSPACES-TOOLCHAINS.md | architecture/project-data.md | Original retained in internal-notes/legacy-source/top-level/PROJECT-MAINTENANCE-WORKSPACES-TOOLCHAINS.md |

| PROJECT_COLLABORATION.md | architecture/project-data.md | Original retained in internal-notes/legacy-source/top-level/PROJECT_COLLABORATION.md |

| PROJECT_IDENTITY.md | reference/design-evolution.md | Original retained in internal-notes/legacy-source/top-level/PROJECT_IDENTITY.md |

| PROJECT_STANCE.md | reference/design-evolution.md | Original retained in internal-notes/legacy-source/top-level/PROJECT_STANCE.md |

| README_IMAGE_PLAN.md | internal-notes/legacy-source/top-level/README_IMAGE_PLAN.md | Original retained in internal-notes/legacy-source/top-level/README_IMAGE_PLAN.md |

| RELEASE_PROCESS.md | engineering/release-and-docs.md | Original retained in internal-notes/legacy-source/top-level/RELEASE_PROCESS.md |

| SAFE_REWORK_MANIFEST.md | reference/documentation-migration.md | Original retained in internal-notes/legacy-source/top-level/SAFE_REWORK_MANIFEST.md |

| SAFE_STATIC_RUNTIME_AND_DIAGNOSTICS_POLICY.md | engineering/build-validation.md | Original retained in internal-notes/legacy-source/top-level/SAFE_STATIC_RUNTIME_AND_DIAGNOSTICS_POLICY.md |

| THEME_RUNTIME_ARCHITECTURE.md | architecture/frontend-and-themes.md | Original retained in internal-notes/legacy-source/top-level/THEME_RUNTIME_ARCHITECTURE.md |

| index.md (former generated landing source) | index.md (curated landing page) | Original retained in internal-notes/legacy-source/top-level/index.md |

Public release articles

The former release-specific articles under docs/articles are preserved under internal-notes/legacy-source/articles. Stable feature behavior was folded into the user guide and architecture pages; patch-version narration is no longer mixed into the timeless documentation.

| Former article | Maintained destination | Preservation |

|---|---|---|

| articles/chat-and-council.md | guide/chat-and-council.md and architecture/council-runtime.md | Original retained in internal-notes/legacy-source/articles/chat-and-council.md |

| articles/embedded-firmware.md | guide/embedded-and-games.md | Original retained in internal-notes/legacy-source/articles/embedded-firmware.md |

| articles/workspaces.md | guide/projects-and-workspaces.md and architecture/project-data.md | Original retained in internal-notes/legacy-source/articles/workspaces.md |

| articles/security.md | architecture/onewire-security.md and architecture/system-overview.md | Original retained in internal-notes/legacy-source/articles/security.md |

| articles/game-director.md | guide/embedded-and-games.md and architecture/council-runtime.md | Original retained in internal-notes/legacy-source/articles/game-director.md |

| articles/documentation.md | guide/documentation.md and engineering/release-and-docs.md | Original retained in internal-notes/legacy-source/articles/documentation.md |

| articles/provider-qualified-council-benchmarking.md | guide/chat-and-council.md and reference/capability-map.md | Original retained in internal-notes/legacy-source/articles/provider-qualified-council-benchmarking.md |

| articles/build-policy-council-recovery.md | engineering/build-validation.md and architecture/council-runtime.md | Original retained in internal-notes/legacy-source/articles/build-policy-council-recovery.md |

| articles/removable-chat-ascii-console.md | guide/embedded-and-games.md | Original retained in internal-notes/legacy-source/articles/removable-chat-ascii-console.md |

Why this structure is safer

- Readers see current behavior first.
- Architecture is grouped by boundary rather than by the order it was learned.
- Important AI Host and Council material remains prominent.

- Local archive paths and source-acquisition notes are not presented as product design.
- Repository archaeology remains possible when a maintainer needs the original context.

reference/index.md

Reference

- Capability map (capability-map.md)
- Design evolution and provenance (design-evolution.md)
- Documentation migration map (documentation-migration.md)
- Complete API reference (../api/index.md)

This section preserves context without turning temporary experiments into current architecture.